

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ  
ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение высшего  
образования «Санкт-Петербургский политехнический университет Петра Великого»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

Направление: 02.03.01 Математика и компьютерные науки

**Лабораторная работа №2 по  
Вычислительной математике**

**Аппроксимация методом наименьших квадратов.  
Вариант 56**

Студент,  
группы 5130201/40003

\_\_\_\_\_ Четвергов И.С

Руководитель,  
доцент к.т.н.

\_\_\_\_\_ Пак В. Г.

«\_\_\_\_\_» \_\_\_\_\_ 2026 г.

Санкт-Петербург  
2026

# Содержание

|          |                                                                  |          |
|----------|------------------------------------------------------------------|----------|
| <b>1</b> | <b>Постановка задачи</b>                                         | <b>3</b> |
| <b>2</b> | <b>Линейная аппроксимация</b>                                    | <b>3</b> |
| 2.1      | Составление вспомогательной таблицы . . . . .                    | 3        |
| 2.2      | Решение нормальной системы . . . . .                             | 4        |
| 2.3      | Вычисление наилучшего среднеквадратического отклонения . . . . . | 4        |
| <b>3</b> | <b>Квадратичная аппроксимация</b>                                | <b>5</b> |
| 3.1      | Дополнительные суммы . . . . .                                   | 5        |
| 3.2      | Решение нормальной системы . . . . .                             | 5        |
| 3.3      | Вычисление наилучшего среднеквадратического отклонения . . . . . | 6        |
| <b>4</b> | <b>Графики и сравнение аппроксимаций</b>                         | <b>7</b> |

# 1 Постановка задачи

Для данной таблицы значений функции построить методом наименьших квадратов линейную и квадратичную полиномиальные модели, вычислить наилучшие среднеквадратические отклонения.

Функция  $y = f(x)$  задана таблицей 1. Требуется построить её аналитическое приближение. Для этого по данным таблицы находится функция  $y = g(x)$ , аппроксимирующая исходную, т.е. удовлетворяющая условию  $g(x_i) \approx y_i, i = 1, \dots, n$ .

Аппроксимирующая функция строится из условия минимума среднеквадратического отклонения:

$$\delta^{(2)}(g) = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - g(x_i))^2} \rightarrow \min.$$

В данной работе аппроксимация проводится в классе алгебраических полиномов степени  $m$ :

$$P_m(x) = a_0 + a_1x + \dots + a_mx^m.$$

Коэффициенты  $a_0, a_1, \dots, a_m$  находятся из нормальной системы МНК:

$$\sum_{j=0}^m a_j \sum_{i=1}^n x_i^{j+k} = \sum_{i=1}^n y_i x_i^k, \quad k = 0, \dots, m.$$

|       |       |       |       |      |      |      |      |      |      |      |
|-------|-------|-------|-------|------|------|------|------|------|------|------|
| $x_i$ | -0.1  | 0.3   | 0.5   | 0.75 | 0.9  | 1.1  | 1.3  | 1.5  | 1.7  | 1.9  |
| $y_i$ | -0.08 | -0.14 | -0.89 | 0.36 | 0.44 | 0.48 | 0.27 | 0.39 | 0.50 | 0.48 |
| $x_i$ | 2.1   | 2.3   | 2.5   | 2.7  | 2.9  | 3.1  | 3.3  | 3.5  | 3.7  | 3.9  |
| $y_i$ | 0.69  | 0.50  | 0.31  | 0.37 | 0.43 | 0.33 | 0.31 | 0.09 | 0.08 | 0.03 |

Таблица 1: Исходные данные функции  $y = f(x)$ .

## 2 Линейная аппроксимация

В этом разделе строится полином наилучшего среднеквадратического отклонения степени  $m = 1$ , то есть линейная аппроксимация  $P_1(x) = a_0 + a_1x$ . Нормальная система МНК для её коэффициентов имеет вид:

$$\begin{cases} a_0 n + a_1 \sum_{i=1}^n x_i = \sum_{i=1}^n y_i, \\ a_0 \sum_{i=1}^n x_i + a_1 \sum_{i=1}^n x_i^2 = \sum_{i=1}^n y_i x_i. \end{cases}$$

### 2.1 Составление вспомогательной таблицы

Для подстановки в нормальную систему вычислим необходимые суммы. На листинге 1 приведён код, вычисляющий суммы:

```
1 import numpy as np
2
3 x = np.array([-0.1, 0.3, 0.5, 0.75, 0.9, 1.1, 1.3, 1.5, 1.7, 1.9,
4               2.1, 2.3, 2.5, 2.7, 2.9, 3.1, 3.3, 3.5, 3.7, 3.9])
5 y = np.array([-0.08, -0.14, -0.89, 0.36, 0.44, 0.48, 0.27, 0.39, 0.50,
6               0.48, 0.69, 0.50, 0.31, 0.37, 0.43, 0.33, 0.31, 0.09,
7               0.08, 0.03])
```

```

8  n = len(x)
9  sx = np.sum(x)
10 sx2 = np.sum(x**2)
11 sy = np.sum(y)
12 sxy = np.sum(x * y)
13
14 print(f"n = {n}")
15 print(f"sum x = {sx:.4f}")
16 print(f"sum x2 = {sx2:.4f}")
17 print(f"sum y = {sy:.4f}")
18 print(f"sum xy = {sxy:.4f}")

```

Листинг 1: Вычисление вспомогательных сумм

## 2.2 Решение нормальной системы

Подставляя вычисленные суммы при  $n = 20$ , получаем нормальную систему для линейной аппроксимации:

$$\begin{cases} 20a_0 + 39.8500a_1 = 4.9500, \\ 39.8500a_0 + 109.1325a_1 = 11.2185. \end{cases}$$

Решаем её с помощью кода, приведённого на листинге 2.

```

1  # Нормальная система: A * a = b
2  A = np.array([[n, sx],
3               [sx, sx2]])
4  b = np.array([sy, sxy])
5
6  a0, a1 = np.linalg.solve(A, b)
7
8  # Аппроксимирующий полином и отклонение
9  P1 = a0 + a1 * x
10 delta1 = np.sqrt(np.sum((y - P1)**2) / n)
11
12 print(f"a0 = {a0:.4f}, a1 = {a1:.4f}")
13 print(f"P1(x) = {a0:.4f} + ({a1:.4f})*x")
14 print(f"delta2(P1) = {delta1:.4f}")

```

Листинг 2: Линейная аппроксимация МНК

Решение нормальной системы даёт коэффициенты:

$$a_0 = 0.1705, \quad a_1 = 0.0386.$$

Таким образом, линейная аппроксимирующая функция имеет вид:

$$P_1(x) = 0.1705 + 0.0386x.$$

## 2.3 Вычисление наилучшего среднеквадратического отклонения

Наилучшее среднеквадратическое отклонение вычисляется по формуле:

$$\delta^{(2)}(P_1) = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - P_1(x_i))^2}$$

Подставляем сумму квадратов отклонений  $\sum_{i=1}^n (y_i - P_1(x_i))^2 = 1.8415$ :

$$\delta^{(2)}(P_1) = \sqrt{\frac{1}{20} \cdot 1.8415} = 0.3034.$$

### 3 Квадратичная аппроксимация

В этом разделе строится полином наилучшего среднеквадратического отклонения степени  $m = 2$ , то есть квадратичная аппроксимация  $P_2(x) = a_0 + a_1x + a_2x^2$ . Нормальная система МНК для её коэффициентов:

$$\begin{cases} a_0n + a_1 \sum x_i + a_2 \sum x_i^2 = \sum y_i, \\ a_0 \sum x_i + a_1 \sum x_i^2 + a_2 \sum x_i^3 = \sum y_i x_i, \\ a_0 \sum x_i^2 + a_1 \sum x_i^3 + a_2 \sum x_i^4 = \sum y_i x_i^2. \end{cases}$$

#### 3.1 Дополнительные суммы

Помимо сумм, найденных в разделе 2, потребуются ещё  $\sum x_i^3$ ,  $\sum x_i^4$  и  $\sum y_i x_i^2$ . Они равны:

$$\sum x_i^3 = 338.2176, \quad \sum x_i^4 = 1121.7825, \quad \sum y_i x_i^2 = 31.9079.$$

#### 3.2 Решение нормальной системы

Подставляя все суммы, нормальная система принимает вид:

$$\begin{cases} 20a_0 + 39.8500a_1 + 109.1325a_2 = 4.9500, \\ 39.8500a_0 + 109.1325a_1 + 338.2176a_2 = 11.2185, \\ 109.1325a_0 + 338.2176a_1 + 1121.7825a_2 = 31.9079. \end{cases}$$

```

1  sx3 = np.sum(x**3)
2  sx4 = np.sum(x**4)
3  sx2y = np.sum(x**2 * y)
4
5  # Нормальная система: A * a = b
6  A = np.array([[n, sx, sx2],
7                [sx, sx2, sx3],
8                [sx2, sx3, sx4]])
9  b = np.array([sy, sxy, sx2y])
10
11 a0, a1, a2 = np.linalg.solve(A, b)
12
13 # Аппроксимирующий полином и отклонение
14 P2 = a0 + a1 * x + a2 * x**2
15 delta2 = np.sqrt(np.sum((y - P2)**2) / n)
16
17 print(f"a0 = {a0:.4f}, a1 = {a1:.4f}, a2 = {a2:.4f}")
18 print(f"P2(x) = {a0:.4f} + ({a1:.4f})*x + ({a2:.4f})*x^2")
19 print(f"delta2(P2) = {delta2:.4f}")

```

Листинг 3: Квадратичная аппроксимация МНК

Решение системы даёт коэффициенты:

$$a_0 = -0.0988, \quad a_1 = 0.4079, \quad a_2 = -0.0926.$$

Квадратичная аппроксимирующая функция:

$$P_2(x) = -0.0988 + 0.4079x - 0.0926x^2.$$

### 3.3 Вычисление наилучшего среднеквадратического отклонения

Наилучшее среднеквадратическое отклонение:

$$\delta^{(2)}(P_2) = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - P_2(x_i))^2}$$

Сумма квадратов отклонений  $\sum_{i=1}^n (y_i - P_2(x_i))^2 = 1.2829$ , поэтому:

$$\delta^{(2)}(P_2) = \sqrt{\frac{1}{20} \cdot 1.2829} = 0.2533.$$

В таблице 2 приведены значения  $P_1(x_i)$ ,  $P_2(x_i)$  и квадраты отклонений для каждого узла.

| $i$    | $x_i$ | $y_i$   | $P_1(x_i)$ | $(y_i - P_1)^2$ | $P_2(x_i)$ | $(y_i - P_2)^2$ |
|--------|-------|---------|------------|-----------------|------------|-----------------|
| 1      | -0.1  | -0.0800 | 0.1667     | 0.0608          | -0.1405    | 0.0037          |
| 2      | 0.3   | -0.1400 | 0.1821     | 0.1037          | 0.0152     | 0.0241          |
| 3      | 0.5   | -0.8900 | 0.1898     | 1.1660          | 0.0820     | 0.9448          |
| 4      | 0.75  | 0.3600  | 0.1995     | 0.0258          | 0.1550     | 0.0420          |
| 5      | 0.9   | 0.4400  | 0.2052     | 0.0551          | 0.1933     | 0.0609          |
| 6      | 1.1   | 0.4800  | 0.2130     | 0.0713          | 0.2378     | 0.0587          |
| 7      | 1.3   | 0.2700  | 0.2207     | 0.0024          | 0.2749     | 0.0000          |
| 8      | 1.5   | 0.3900  | 0.2284     | 0.0261          | 0.3046     | 0.0073          |
| 9      | 1.7   | 0.5000  | 0.2361     | 0.0696          | 0.3268     | 0.0300          |
| 10     | 1.9   | 0.4800  | 0.2438     | 0.0558          | 0.3417     | 0.0191          |
| 11     | 2.1   | 0.6900  | 0.2515     | 0.1923          | 0.3491     | 0.1162          |
| 12     | 2.3   | 0.5000  | 0.2593     | 0.0580          | 0.3491     | 0.0228          |
| 13     | 2.5   | 0.3100  | 0.2670     | 0.0019          | 0.3416     | 0.0010          |
| 14     | 2.7   | 0.3700  | 0.2747     | 0.0091          | 0.3267     | 0.0019          |
| 15     | 2.9   | 0.4300  | 0.2824     | 0.0218          | 0.3043     | 0.0158          |
| 16     | 3.1   | 0.3300  | 0.2901     | 0.0016          | 0.2745     | 0.0031          |
| 17     | 3.3   | 0.3100  | 0.2978     | 0.0001          | 0.2373     | 0.0053          |
| 18     | 3.5   | 0.0900  | 0.3056     | 0.0465          | 0.1926     | 0.0105          |
| 19     | 3.7   | 0.0800  | 0.3133     | 0.0544          | 0.1405     | 0.0037          |
| 20     | 3.9   | 0.0300  | 0.3210     | 0.0847          | 0.0810     | 0.0026          |
| СУММЫ: |       |         |            | 1.8415          |            | 1.2829          |

Таблица 2: Значения аппроксимирующих полиномов и квадраты отклонений.

## 4 Графики и сравнение аппроксимаций

Для наглядного сравнения исходных данных с построенными моделями построим графики. На листинге 4 приведён соответствующий код.

```
1 import matplotlib.pyplot as plt
2
3 a0_lin = 0.1705; a1_lin = 0.0386
4 a0_quad = -0.0988; a1_quad = 0.4079; a2_quad = -0.0926
5
6 x_fine = np.linspace(-0.2, 4.0, 300)
7 P1_fine = a0_lin + a1_lin * x_fine
8 P2_fine = a0_quad + a1_quad * x_fine + a2_quad * x_fine**2
9
10 plt.figure(figsize=(8, 5))
11 plt.scatter(x, y, color='blue', zorder=5, label='Исходные данные')
12 plt.plot(x_fine, P1_fine, color='black', linewidth=1.5,
13         label=r'$P_1(x)$ линейная')
14 plt.plot(x_fine, P2_fine, color='red', linewidth=1.5, label=r'$P_2(x)$
15         квадратичная')
16 plt.xlabel('x')
17 plt.ylabel('y')
18 plt.title('Аппроксимация методом наименьших квадратов')
19 plt.legend()
20 plt.grid(True)
21 plt.tight_layout()
22 plt.savefig('report/pct/approximation.png', dpi=150)
23 plt.show()
```

Листинг 4: Построение графиков аппроксимаций

На рисунке 1 изображены исходные данные и обе аппроксимирующие кривые. Видно, что квадратичная модель  $P_2(x)$  лучше описывает нелинейный тренд перепадов табличных данных по сравнению с линейной  $P_1(x)$ .

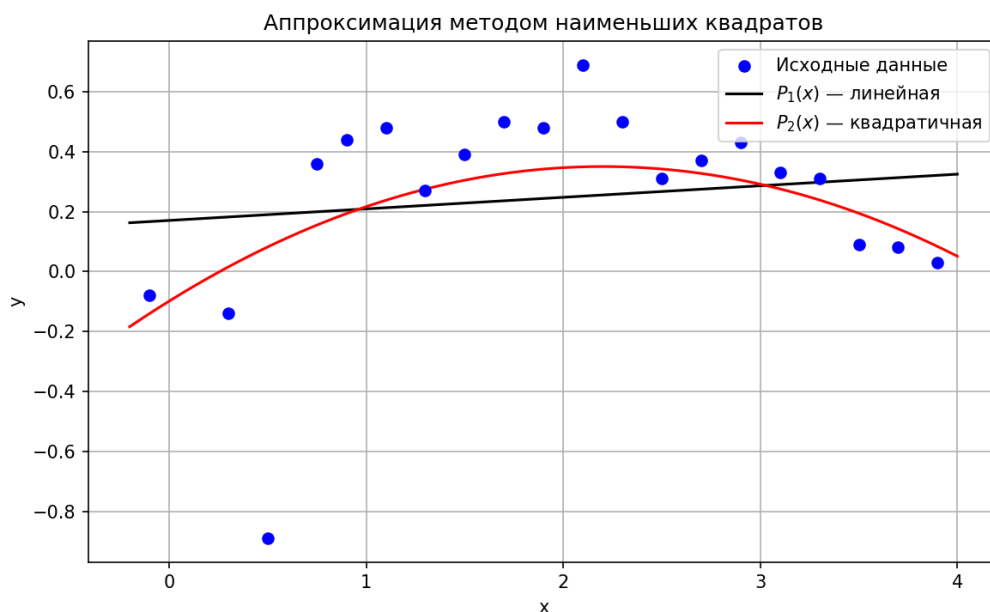


Рис. 1: Графики исходных данных и аппроксимирующих полиномов.

## Заключение

В ходе выполнения лабораторной работы была изучена полиномиальная аппроксимация методом наименьших квадратов. По таблице из  $n = 20$  узлов были построены два аппроксимирующих полинома.

Для линейной аппроксимации  $P_1(x) = a_0 + a_1x$  нормальная система МНК была составлена и решена, в результате чего получены коэффициенты  $a_0 = 0.1705$  и  $a_1 = 0.0386$ :

$$P_1(x) = 0.1705 + 0.0386x.$$

Наилучшее среднеквадратическое отклонение составило  $\delta^{(2)}(P_1) = 0.3034$ .

Для квадратичной аппроксимации  $P_2(x) = a_0 + a_1x + a_2x^2$  получены коэффициенты  $a_0 = -0.0988$ ,  $a_1 = 0.4079$ ,  $a_2 = -0.0926$ :

$$P_2(x) = -0.0988 + 0.4079x - 0.0926x^2.$$

Наилучшее среднеквадратическое отклонение составило  $\delta^{(2)}(P_2) = 0.2533$ .

Квадратичная модель точнее линейной:  $\delta^{(2)}(P_2) < \delta^{(2)}(P_1)$ . Это закономерно, поскольку полином более высокой степени обладает большей гибкостью при подгонке к данным. Все вычисления выполнены программно на языке Python с использованием библиотеки NumPy.